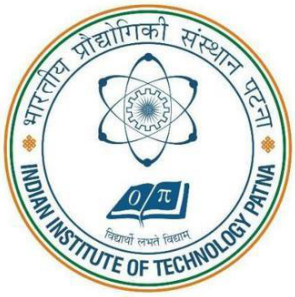
Indian Institute of Technology Patna



Department of Computer Science & Engineering

CS5103 Computing Lab

Agentic AI Assignment – Phase 1 Report

Topic : AutoGrader – A Rule-Driven Evaluator Agent for C Programs

Name: Shakti Prasad Nahak Roll Number: 2611AI46

Name: Akash Prasad Roll Number: 2611AI08

Name: Gourav Rajak Roll Number: 2611AI07

Name: Bibaswan Das Roll Number: 2611MC16

Course In-charge

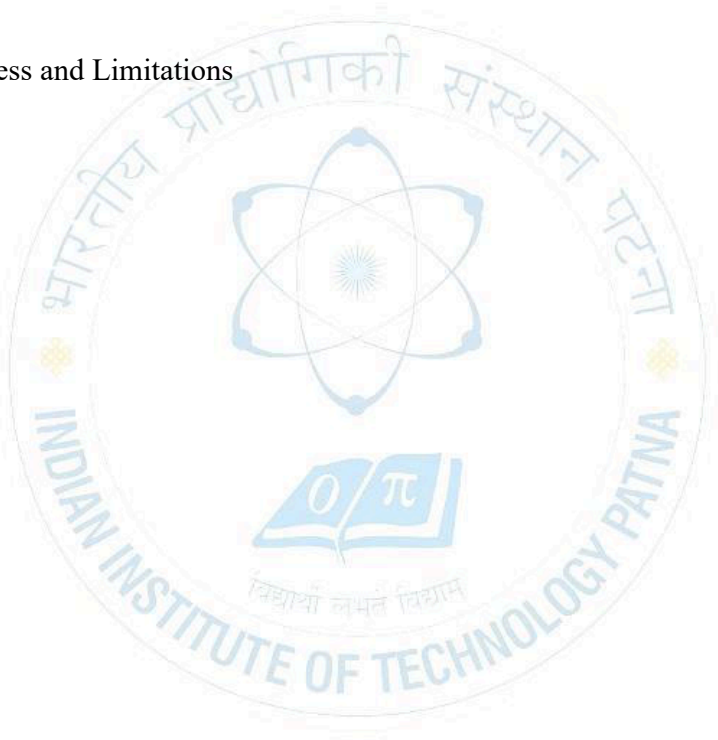
Prof. Arijit Mondal

Department of Computer Science & Engineering

IIT Patna

Table of Contents

1. Aim of the Project
2. Objectives
3. Proposed Architecture
4. Grading Pipeline
5. Metrics and Rules File
6. Design Decisions
7. Work Done in Phase 1
8. Distribution of Work
9. Plan and Timeline
10. Expected Usefulness and Limitations
11. References



1. Aim of the Project :

To build an agent that takes a student's C submission and a set of grading rules written by the instructor, runs the submission safely, checks it against every rule, and returns one overall grade with feedback that shows where the program falls short. The same submission graded under the same rules must always receive the same grade.

2. Objectives :

- Run a C submission safely under fixed time and memory limits, and survive programs that crash or never halt.
- Read the metrics, thresholds and weights from a rules file kept outside the code.
- Evaluate each metric with its own analyzer and combine the results into one weighted grade.
- For an incorrect program, report a specific failing input with the expected output beside it, and flag memory errors and leaks.
- Flag suspected copying from classmates, online sources or LLMs.
- State the usefulness and the limitations of the agent with concrete examples.

3. Proposed Architecture :

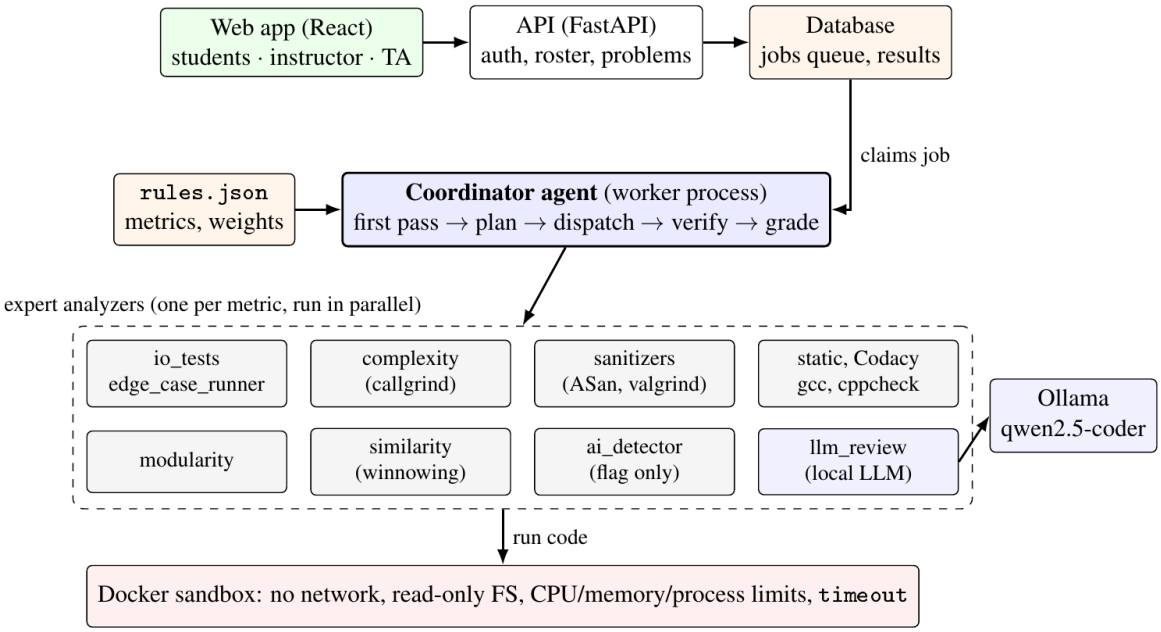


Figure 1: Proposed architecture. The web server never runs student code; a separate worker runs the coordinator agent, which dispatches one analyzer per metric inside the sandbox.

The **coordinator agent** decides which analyzers to run, collects their scores and computes the grade. Each **expert analyzer** handles one metric with its own tool. Programs are executed only inside a **Docker sandbox** with no network, a read-only file system and CPU, memory and process

limits. A **local language model** (served by Ollama) is used only for readability review and plain-language feedback.

4. Grading Pipeline :

1. **First pass:** compile with `-Wall -Wextra`, run the sample input once and check that `main`, input and output exist.
2. **Plan:** if the program does not compile or hangs, the analyzers that need to run it are skipped and the rest still run.
3. **Dispatch:** the analyzers named in `rules.json` run in parallel, each in its own sandbox.
4. **Verify:** test inputs suggested by the LLM count only if running them against the reference solution confirms a failure.
5. **Grade:** grade = Σ $\text{score}_i \times \text{weight}_i \times 100$, computed in code; the LLM never decides correctness.

5. Metrics and Rules File :

Metric	Analyzer	Method	Weight
correctness	io_tests	hidden test cases, outputs compared	0.20
syntax	compiler_warnings	gcc -Wall -Wextra, penalty per warning	0.10
structure	static_analysis	cyclomatic complexity, function length, cppcheck	0.10
modularity	function_split	call graph from main	0.10
logical	edge_case_runner	edge cases and seeded random inputs vs. reference	0.10
complexity	complexity_estimator	valgrind instruction counts fitted to a growth class	0.10
memory_safety	sanitizer_run	AddressSanitizer, UBSan, valgrind leak check	0.10
readability	llm_review	rubric prompt to a local model	0.10
originality	similarity_flag	winnowing fingerprints, threshold 0.85	0.10

The instructor edits only `rules.json` to add a metric or change a weight or threshold; the agent's code does not change. A hash of the rules file is stored with every grade.

6. Design Decisions :

- **Execution decides, the LLM advises.** Correctness, memory safety and speed come from running the program. A pre-trained code model with a fixed rubric, temperature 0 and a fixed seed is enough for readability feedback, so no fine-tuning is planned.
- **Reproducibility.** Seeded test generation, instruction counts instead of wall-clock time, and cached LLM replies.
- **Local models.** Student code does not leave the machine; the model runs through Ollama.

- **Originality.** MOSS-style winnowing against classmates, known web solutions and LLM-generated solutions. Matches are flags for the instructor, not automatic penalties.

7. Work Done in Phase 1 :

- Studied the assignment and mapped each requirement to a component of the architecture.
- Compared ways of running untrusted code and selected Docker as the sandbox.
- Tried gcc warnings, cppcheck, AddressSanitizer, UBSan and valgrind on small buggy C programs and decided which metric each tool serves.
- Studied the use of LLMs for code review and selected a local code model through Ollama.
- Studied winnowing for copy detection and planned the three comparison corpora.
- Chose the technology stack (Python with FastAPI, a separate worker process, SQLite, React) and set up the development environment and a first sample problem.

8. Distribution of Work :

Member	Responsibility
Shakti Prasad Nahak	Coordinator agent, rules format, sandbox, job queue, integration and testing
Akash Prasad	Execution analyzers: I/O tests, edge cases and counter-examples, complexity estimation
Gourav Rajak	Code-quality analyzers: compiler warnings, static analysis, modularity, sanitizers
Bibaswan Das	LLM review, originality and AI-origin detection, web interface

9. Plan and Timeline :

Period	Work
10 – 16 Oct	Requirement study and research on tools (done)
17 – 22 Oct	Architecture, rules format, technology choice, environment setup (done)
23 – 29 Oct	Coordinator, sandbox and all analyzers; first end-to-end grade on sample problems
30 Oct – 5 Nov	Web interface, testing with malicious programs, final report, GitHub repository, video

10. Expected Usefulness and Limitations :

Usefulness: grading a whole lab unattended with explainable per-metric marks; showing each student the exact input that breaks the program; narrowing plagiarism review to flagged pairs; screening submissions in coding tests.

Limitations:

- Tests cannot prove correctness; a bug that no test or generated input triggers still gets full marks.

- An LLM's readability score can change with the model or hardware, so replies must be cached.
- Similarity on very short programs is unreliable, because every correct short answer looks alike.
- An AI-origin signal can flag a careful human writer, so it should never change the grade by itself.

11. References :

1. CS5103 Agentic AI assignment: AutoGrader, a rule-driven evaluator for C programs.
2. S. Schleimer, D. Wilkerson, A. Aiken. Winnowing: local algorithms for document fingerprinting. SIGMOD 2003.
3. Docker, Valgrind, AddressSanitizer, cppcheck and Ollama documentation.

भारतीय प्रौद्योगिकी संस्थान पटना
INDIAN INSTITUTE OF TECHNOLOGY PATNA